



Pre-Modification System Testing - CodeSmile

Autore	Matricola
Daniele Pio Scaparra	NF22500101
Grazia Bonagura	NF22500105
Antonio Nappi	NF22500200

Indice

1. Introduzione.....	2
1.1. Contesto e Motivazione.....	2
1.2. Scope del Testing.....	2
1.3. Obiettivi del Testing.....	2
2. Metodologia.....	2
2.1. Approccio Black-Box.....	2
2.2. Category Partition.....	3
2.3. Weak Equivalence Class Testing.....	3
3. Funzionalità Identificate.....	3
4. F1 - Static Analysis CLI.....	3
4.1. Specifica Black-Box.....	3
4.2 Category Partition – Static Analysis CLI (F1).....	4
4.3 Test Case – Static Analysis CLI (F1).....	5
5. F2 – Report Generation.....	7
5.1 Specifica Black-Box.....	7
5.2 Category Partition – Report Generation (F2).....	8
5.3 Test Case – Report Generation (F2).....	10

1. Introduzione

1.1. Contesto e Motivazione

Il presente documento descrive l'attività di **Pre-Modification System Testing** svolta sul tool **CodeSmile**, con l'obiettivo di caratterizzare il comportamento del sistema **prima** dell'applicazione delle Change Request approvate nella fase preliminare del progetto.

Le Change Request considerate sono le seguenti:

- **CR1**: Miglioramento del detector *In-Place APIs Misused* (manutenzione correttiva/perfettiva)
- **CR2**: Creazione del call graph del progetto (manutenzione evolutiva)
- **CR3**: Analisi e visualizzazione del call graph (manutenzione evolutiva)

1.2. Scope del Testing

In accordo con lo scope delle Change Request, l'attività di testing è limitata alle sole componenti direttamente coinvolte dalle modifiche previste.

Componenti **IN SCOPE**:

- CLI (cli/cli_runner.py, components/project_analyzer.py, components/inspector.py)
- Detection Rules (detection_rules/, con focus su generic/in_place_apis_misused.py)
- Report Generation (report/report_generator.py)
- Code Extractors (code_extractor/*)

Componenti **OUT OF SCOPE**:

- AI-Based Detection (finetuning/, data_preparation/)
- Web Application (webapp/)
- GUI (gui/code_smell_detector_gui.py, gui/textbox_redirect.py)

1.3. Obiettivi del Testing

1. **Baseline Establishment**: Stabilire comportamento attuale del sistema come riferimento per regression testing post-CR
 2. **Risk Assessment**: Identificare componenti critiche impattate dalle CR
 3. **Test Frame Generation**: Produrre set sistematico di test frame per validare stato pre-modifica
 4. **Gap Analysis**: Identificare eventuali lacune nella suite di testing esistente
 5. **Test Reusability**: Generare test frame riutilizzabili come regression suite post-CR
-

2. Metodologia

2.1. Approccio Black-Box

Il sistema è analizzato adottando un **approccio black-box**: i test sono progettati considerando

esclusivamente le specifiche funzionali, gli input osservabili, gli output prodotti e il comportamento esterno del sistema, senza fare riferimento all'implementazione interna.

2.2. Category Partition

Per la progettazione dei test è stato adottato il **Category Partition Method**, seguendo il seguente processo sistematico:

1. Identificazione delle funzionalità rilevanti nello scope
2. Identificazione dei parametri (input, output e condizioni ambientali)
3. Definizione delle categorie associate a ciascun parametro
4. Partizionamento delle categorie in scelte
5. Definizione dei vincoli tra le scelte

2.3. Weak Equivalence Class Testing

I test cases sono stati generati applicando il criterio di **Weak Equivalence Class Testing**, secondo cui:

- viene definito un **test frame valido** contenente esclusivamente scelte valide (baseline scenario);
- vengono definiti **N test frame invalidi**, ciascuno contenente una sola scelta invalida, mantenendo valide tutte le altre.

Il numero *totale di test cases* è dato dalla formula: $1 + \sum (\text{scelte invalide per parametro})$

3. Funzionalità Identificate

ID	Funzionalità	Descrizione Black-Box	Componente
F1	Static Analysis CLI	Analisi progetto Python via command line → CSV risultati	cli/, components/
F2	Report Generation	Aggregazione risultati CSV → report Excel/PNG	report/

4. F1 - Static Analysis CLI

4.1. Specifica Black-Box

L'analisi statica tramite CLI viene invocata mediante il seguente comando:

```
python -m cli.cli_runner --input <PATH> --output <PATH> [--parallel]
```

```
[--max_walkers N] [--resume] [--multiple]
```

4.2 Category Partition – Static Analysis CLI (F1)

C1 – Input Path (--input)

ID	Scelta	Tipo
C1.1	Directory esistente contenente ≥ 1 file .py	Valida
C1.2	Directory esistente senza file .py	Valida
C1.3	Path inesistente	Invalida
C1.4	Path che punta a file (non directory)	Invalida
C1.5	Directory senza permessi di lettura	Invalida

C2 – Output Path (--output)

ID	Scelta	Tipo
C2.1	Directory esistente e scrivibile	Valida
C2.2	Directory non esistente ma creabile	Valida
C2.3	Directory senza permessi di scrittura	Invalida
C2.4	Path che punta a file	Invalida

C3 – Modalità di Esecuzione

ID	Scelta	Tipo
C3.1	Esecuzione sequenziale (flag --parallel assente)	Valida
C3.2	Esecuzione parallela (flag --parallel presente)	Valida

C4 – Numero di Worker (--max_walkers)

ID	Scelta	Tipo	Condizione
C4.1	Parametro omissso (default)	Valida	sempre
C4.2	Valore intero > 0	Valida	se C3.2

C4.3	Valore = 0	Invalida	se C3.2
C4.4	Valore < 0	Invalida	se C3.2
C4.5	Valore non numerico	Invalida	se C3.2

C5 – Resume (--resume)

ID	Scelta	Tipo
C5.1	Flag assente	Valida
C5.2	Flag presente con checkpoint valido	Valida
C5.3	Flag presente senza checkpoint	Invalida

C6 – Multiple Projects (--multiple)

ID	Scelta	Tipo
C6.1	Flag assente (singolo progetto)	Valida
C6.2	Flag presente con directory contenente più progetti	Valida
C6.3	Flag presente con singolo progetto	Invalida

C7 – Stato File System (condizioni ambientali)*

ID	Scelta	Tipo
C7.1	File system accessibile e stabile	Valida
C7.2	File bloccati / accesso concorrente	Invalida
C7.3	Spazio disco insufficiente	Invalida

*** Nota sulle condizioni ambientali (C7).**

Le condizioni ambientali sono state incluse nel Category Partition in quanto, pur non costituendo input espliciti dell'utente, incidono direttamente sul comportamento osservabile del sistema, come riconosciuto dalla letteratura sul black-box testing.

4.3 Test Case – Static Analysis CLI (F1)

I test case sono stati derivati applicando il criterio di **Weak Equivalence Class Testing** ai test frame definiti per la funzionalità F1.

Ogni test case è descritto tramite **un identificativo univoco**, il **test frame** (insieme delle scelte selezionate) e l'**oracolo atteso**, espresso in termini di comportamento osservabile del sistema.

Test Case ID	Test Frame	Oracolo Atteso
TC-F1-01	C1.1, C2.1, C3.1, C4.1, C5.1, C6.1, C7.1	L'analisi parte in modalità sequenziale e vengono generati i risultati CSV nell'output senza errori.
TC-F1-02	C1.3, C2.1, C3.1, C4.1, C5.1, C6.1, C7.1	Errore: input path inesistente. Nessuna analisi avviata.
TC-F1-03	C1.4, C2.1, C3.1, C4.1, C5.1, C6.1, C7.1	Errore: input non è una directory. Nessuna analisi avviata.
TC-F1-04	C1.5, C2.1, C3.1, C4.1, C5.1, C6.1, C7.1	Errore: directory di input non leggibile (permessi). Nessuna analisi avviata.
TC-F1-05	C1.1, C2.3, C3.1, C4.1, C5.1, C6.1, C7.1	Errore: output non scrivibile (permessi). Nessun risultato prodotto.
TC-F1-06	C1.1, C2.4, C3.1, C4.1, C5.1, C6.1, C7.1	Errore: output path non è una directory valida (punta a file). Nessun risultato prodotto.
TC-F1-07	C1.1, C2.1, C3.2, C4.3, C5.1, C6.1, C7.1	Errore: --max_walkers = 0 non valido. Analisi non avviata.
TC-F1-08	C1.1, C2.1, C3.2, C4.4, C5.1, C6.1, C7.1	Errore: --max_walkers negativo non valido. Analisi non avviata.
TC-F1-09	C1.1, C2.1, C3.2, C4.5, C5.1, C6.1, C7.1	Errore: --max_walkers non numerico. Analisi non avviata.
TC-F1-10	C1.1, C2.1, C3.1, C4.1, C5.3, C6.1, C7.1	Errore: --resume attivo senza checkpoint valido. Ripresa non possibile.

TC-F1-11	C1.1, C2.1, C3.1, C4.1, C5.1, C6.3, C7.1	Errore: --multiple usato su input che non contiene più progetti (violazione vincolo).
TC-F1-12	C1.1, C2.1, C3.1, C4.1, C5.1, C6.1, C7.2	Errore di I/O durante l'analisi dovuto a lock/accesso concorrente: l'analisi fallisce o viene interrotta con messaggio di errore.
TC-F1-13	C1.1, C2.1, C3.1, C4.1, C5.1, C6.1, C7.3	Errore di I/O: spazio disco insufficiente. L'analisi viene interrotta e l'output non viene completato.

Nota di copertura

La suite di test sopra definita comprende:

- **1 test case valido** (TC-F1-01), rappresentativo del flusso nominale;
- **12 test case invalidi**, ciascuno caratterizzato da una singola scelta invalida, in accordo con il criterio di **Weak Equivalence Class Testing**.

5. F2 – Report Generation

5.1 Specifica Black-Box

Il modulo di generazione dei report viene invocato tramite il seguente comando:

```
python -m report.report_generator --input <PATH> --output <PATH>
```

Il sistema:

1. valida i parametri di input e output;
2. carica i file CSV presenti nella directory di input;
3. presenta un **menu interattivo** per la selezione del tipo di report;
4. genera i file di output corrispondenti alla scelta effettuata.

5.2 Category Partition – Report Generation (F2)

R1 – Input path (--input)

ID	Scelta	Tipo
R1.1	Directory esistente che non contiene la sottocartella project_details/	Invalida
R1.2	Directory project_details/ presente ma senza alcun file CSV	Invalida
R1.3	Directory project_details/ contenente un file CSV vuoto (size = 0, nessuna colonna)	Invalida
R1.4	Directory project_details/ contenente almeno un file CSV non vuoto	Valida

R2 – Contenuto del CSV caricato

ID	Scelta	Tipo
R2.1	CSV parsabile con schema completo CodeSmile (filename, function_name, smell_name, line, description, additional_info)	Valida
R2.2	CSV parsabile ma con schema non conforme (colonne mancanti)	Invalida

R3 – Selezione del Report (menu interattivo)

ID	Scelta	Tipo	Condizione
R3.1	General Smell Report (opzione 1)	Valida	se R1.4
R3.2	Project-Specific Report (opzione 2)	Valida	se R1.4
R3.3	Both Reports (opzione 3)	Valida	se R1.4
R3.4	Plot Report (opzione 4)	Valida	se R1.4 e R2.1
R3.5	Summary .xlsx Report (opzione 5)	Valida	se R1.4
R3.6	Exit (opzione 6)	Valida	se R1.4
R3.7	Valore non previsto ($\neq$ 1–6)	Invalida	se R1.4

R4 – Output Path (--output)

ID	Scelta	Tipo
R4.1	Directory esistente e scrivibile	Valida
R4.2	Directory non esistente ma creabile	Valida

R4.3	Directory non scrivibile	Invalida
R4.4	Path che punta a file	Invalida

R5 – Stato File System (condizioni ambientali)*

ID	Scelta	Tipo
R5.1	File system accessibile e stabile	Valida
R5.2	File system in errore/lock durante la scrittura	Invalida
R5.3	Spazio disco insufficiente	Invalida

*vedi sezione 4.2

5.3 Test Case – Report Generation (F2)

I test case sono stati derivati applicando il criterio di **Weak Equivalence Class Testing** ai test frame definiti per la funzionalità F2.

Ogni test case è descritto tramite un identificativo univoco, il **test frame** (insieme delle scelte selezionate) e l'**oracolo atteso**, espresso in termini di comportamento osservabile del sistema.

Test Case ID	Test Frame	Oracolo Atteso
TC-F2-01	R1.4, R2.1, R3.1, R4.1, R5.1	Il menu viene visualizzato e il General Smell Report viene generato correttamente nella directory di output senza errori.

TC-F2-02	R1.1, R4.1, R5.1	Errore: la directory di input non contiene la sottocartella project_details/. Il programma termina senza presentare il menu.
TC-F2-03	R1.2, R4.1, R5.1	Errore: nessun file CSV presente nella directory project_details/. Il programma termina senza presentare il menu.
TC-F2-04	R1.3, R4.1, R5.1	Errore: file CSV non parsabile (CSV vuoto). Il programma termina senza presentare il menu.
TC-F2-05	R1.4, R2.2, R4.1, R5.1	Errore: schema CSV non conforme (colonne mancanti). Nessun report viene generato; l'esecuzione termina prima della selezione
TC-F2-06	R1.4, R2.1, R3.7, R4.1, R5.1	Errore: scelta non valida nel menu. Il programma termina senza generare alcun report.
TC-F2-07	R1.4, R2.1, R3.1, R4.3, R5.1	Errore: directory di output non scrivibile. Nessun report viene generato.
TC-F2-08	R1.4, R2.1, R3.1, R4.4, R5.1	Errore: output path non valido (punta a un file). Nessun report viene generato.
TC-F2-09	R1.4, R2.1, R3.1, R4.1, R5.2	Errore di I/O durante la generazione del report dovuto a lock/errore del file system. Il report non viene completato.
TC-F2-10	R1.4, R2.1, R3.1, R4.1, R5.3	Errore di I/O: spazio disco insufficiente durante la scrittura. Il file di output non viene generato o risulta incompleto.

Nota di copertura

La suite di test sopra definita comprende:

- **1 test case valido** (TC-F2-01);
- **9 test case invalidi**, ciascuno caratterizzato da una singola scelta invalida o dalla violazione di un vincolo specifico,

in accordo con il criterio di **Weak Equivalence Class Testing**.